

4 AI Projects

Resume + Interview Blueprint

Step-by-step build plan · Real tools · Real links · Interview Q&A; · No fake data

01 Multi-Agent AI System

02 AI Workflow Automation Platform

03 LLM-Based Intelligent Assistant

04 AI Safety & Evaluation Dashboard

Each section: Overview · Build Steps · Tools & Links · Resume Bullets · Interview Q&A;

01 — Multi-Agent AI System

Multiple AI agents communicate, divide tasks, and solve complex problems together. One of the hottest AI skills demanded in 2025 job postings.

OVERVIEW

What it is	A system where multiple specialised AI agents work together — one searches the web, one writes, one reviews. They share state and pass outputs to each other.
Why it matters	Agentic AI is the #1 demanded skill in AI engineering roles. Shows you understand orchestration, not just single API calls.
Difficulty	Intermediate — 2 to 3 weeks for a working demo.
Core concept	Each agent = a function that reads shared state, does work, writes back. A graph defines which agent runs next and under what condition.

STEP-BY-STEP BUILD PLAN

Step 1	Install & Setup	Install Python 3.11+, a multi-agent framework, and an LLM SDK. Create a virtual environment. Store API keys in a .env file.
Step 2	Define Your 3 Agents	Researcher — searches the web for real-time info. Writer — drafts content based on research. Critic — reviews the draft and gives a score with feedback.
Step 3	Build the State Graph	Use a StateGraph to define nodes (agents) and edges (flow). Each node receives the shared state dict, processes it, and returns an updated state.
Step 4	Add Memory	Store conversation history in a short-term memory object so agents remember what previous steps produced without re-running them.
Step 5	Add Conditional Routing	If the Critic score is below 7/10, route back to the Writer for revision. Add a max_iterations=3 limit to prevent infinite loops.
Step 6	Build a Simple UI	Use a lightweight Python UI library to show each agent's output in real time as the graph runs — users see Researcher → Writer → Critic happening live.
Step 7	Deploy & Document	Deploy on a free hosting platform. Draw an architecture diagram using a free tool. Record a 2-minute video showing the system solving a real problem.

TOOLS & REAL LINKS

Tool / Framework	Purpose	Link (type in browser)
LangGraph	Multi-agent graph orchestration framework	https://langchain-ai.github.io/langgraph/
LangChain	LLM chaining, memory, tool use	https://python.langchain.com/docs/introduction/
Tavily Search API	Real-time web search tool for agents	https://tavily.com/

LangGraph tutorials	Step-by-step agent building guides	https://langchain-ai.github.io/langgraph/tutorials/
Streamlit	Fast Python UI to demo the agent	https://streamlit.io/
draw.io (diagrams.net)	Free architecture diagram tool	https://app.diagrams.net/
Hugging Face Spaces	Free deployment for your demo	https://huggingface.co/spaces

RESUME BULLET POINTS

- Built a multi-agent AI system with 3 specialised agents (Researcher, Writer, Critic) that autonomously complete complex tasks end-to-end using a graph-based orchestration framework.
- Implemented stateful agent communication where agents share a structured state dict, enabling context-aware handoffs and conditional re-routing based on quality scores.
- Deployed the system with a live UI showing real-time agent activity, and documented the architecture with a diagram and a 2-minute walkthrough video.

INTERVIEW Q&A;

Q: How do the agents communicate in your system?

A: Each agent is a node in a graph. They communicate through a shared state dictionary. One agent writes its output into the state, the next reads from it. The graph edges define the order and conditions for transitions between agents.

Q: How did you handle cases where an agent gives a wrong or low-quality output?

A: I added a Critic agent that scores the Writer's output from 1 to 10 against a rubric. If the score is below 7, the graph routes back to the Writer with the feedback as context. I also added a maximum iteration limit to prevent infinite loops.

Q: Why use a multi-agent approach instead of one big prompt?

A: Separation of concerns. Each agent is specialised and can be tested independently. A single massive prompt becomes uncontrollable and expensive. With agents, you can swap, upgrade, or debug one node without breaking the whole system.

02 — AI Workflow Automation Platform

Chain AI models with external services to automate real business tasks. Directly reduces cost — companies pay premium for this skill.

OVERVIEW

What it is	A platform where you visually connect triggers, AI reasoning steps, and actions — like auto-classifying emails, enriching leads, or extracting invoice data.
Why it matters	Every company wants to automate repetitive work. Knowing visual automation + AI = delivering this in hours instead of months.
Difficulty	Beginner-friendly — visual drag-and-drop builder, no heavy coding required.
Core concept	Trigger → AI node (reasons about data) → Action. The AI node decides what to do next based on the input it receives.

STEP-BY-STEP BUILD PLAN

Step 1	Set Up Your Automation Tool	Create a free account on a visual workflow tool (see links below). Explore the canvas — every workflow has a trigger node and action nodes.
Step 2	Build Workflow 1 — Email Classifier	Trigger: new email arrives. AI node: classify as Sales / Support / Spam with a short explanation. Action: route to the right folder, spreadsheet row, or team chat channel.
Step 3	Build Workflow 2 — Lead Enricher	Trigger: new row added to a spreadsheet (name + company). AI node: write a 3-sentence summary of what this company does. Action: write the enriched data back to a new column automatically.
Step 4	Build Workflow 3 — Document Data Extractor	Trigger: new PDF uploaded to cloud storage. AI node: extract key fields — date, amount, vendor name — as structured data. Action: save extracted fields to a database or spreadsheet row.
Step 5	Connect Your LLM	In the AI node, add your API key. Give the model a clear system prompt and tell it to return structured output (JSON) so downstream steps can parse it reliably.
Step 6	Add Error Handling	Add an error trigger node — if any step fails, send a notification to your phone or email. Add a retry setting on HTTP steps (3 retries with 5-second delay).
Step 7	Deploy & Document	Activate all 3 workflows. Export them as JSON and upload to a GitHub repo. Write a one-page case study per workflow: Problem, Solution, Estimated Time Saved.

TOOLS & REAL LINKS

Tool / Framework	Purpose	Link (type in browser)
n8n (recommended)	Open-source visual automation with native AI Agent node	https://n8n.io/

n8n AI Agent guide	Official guide to building AI agents in n8n	https://docs.n8n.io/advanced-ai/intro-tutorial/
Make (alternative)	Visual automation platform with 1000+ integrations	https://www.make.com/en
Zapier (alternative)	Beginner-friendly automation platform	https://zapier.com/
Airtable	Database for storing automation outputs	https://airtable.com/developers/web/api/introduction
Google Sheets API	Trigger automations from spreadsheet changes	https://developers.google.com/sheets/api/guides/concepts

RESUME BULLET POINTS

- Designed and deployed 3 AI-powered automation workflows that automate email classification, lead enrichment, and invoice data extraction — replacing manual repetitive tasks entirely.
- Connected AI reasoning steps with external APIs (spreadsheets, cloud storage, databases) using a visual workflow builder with an AI Agent node that decides which actions to take.
- Built reusable workflow templates (exported as JSON), documented each automation with a business case and estimated time savings for portfolio presentation.

INTERVIEW Q&A;

Q: How is an AI workflow different from a regular automation?

A: A regular automation follows a fixed if-this-then-that rule. An AI workflow adds a reasoning step — the model reads the data and decides what category it is, what to extract, or what response to generate. This handles variability that rigid rules can't.

Q: How did you handle errors in your automations?

A: I added an error trigger node connected to a notification. I also added retry logic on HTTP steps and validated AI output with a schema check before writing to the database — so a badly formatted AI response doesn't corrupt data.

Q: Can you estimate the ROI of one of your workflows?

A: My email classifier handled around 200 emails per day. Manual sorting takes roughly 30 minutes. The automation runs in under 2 minutes. Over a month that is roughly 10 hours of saved time per person.

03 — LLM-Based Intelligent Assistant

A full-stack AI assistant with memory, tool use, and RAG over your own documents. Shows real LLM engineering — not just API calls.

OVERVIEW

What it is	A production-grade chatbot that remembers conversation history, can search the web or do calculations, and answers from your own uploaded documents.
Why it matters	Every company building AI products needs engineers who can ship real LLM apps — not just call a chat API and display the result.
Difficulty	Intermediate-Advanced — 3 to 4 weeks for full features.
Core concept	Three layers: (1) LLM for reasoning, (2) Vector DB for document retrieval, (3) Tool use so the model can take actions, not just talk.

STEP-BY-STEP BUILD PLAN

Step 1	Backend Setup	Create a Python API server with a /chat endpoint. Accept {message, session_id} as input. Return streaming responses so text appears word by word, not all at once.
Step 2	Add Conversation Memory	Store each conversation's history in a dictionary keyed by session_id. Pass the last 10 messages as context in every API call so the model remembers what was said.
Step 3	Build RAG — Document Upload	Let users upload PDF or text files. Split them into chunks of ~500 tokens. Convert each chunk to an embedding vector using an embedding model. Store the vectors in a vector database.
Step 4	Build RAG — Retrieval	On each user question, embed the question and find the top 5 most similar chunks. Inject those chunks into the prompt as context. The model answers from real documents, not from hallucinated memory.
Step 5	Add Tool Use (Function Calling)	Define tools: web_search, calculator, get_current_date. Pass tool definitions to the model. The model decides when to call a tool, you run it, send the result back, and the model uses it in its answer.
Step 6	Build the Frontend	React or plain HTML. Chat interface with streaming message display. Show a visual indicator when the model is calling a tool (e.g., 'Searching the web...').
Step 7	Evaluate & Deploy	Use an evaluation library to measure answer quality: faithfulness, relevancy, context recall. Deploy backend and frontend on free hosting platforms. Link in your GitHub README.

TOOLS & REAL LINKS

Tool / Framework	Purpose	Link (type in browser)
FastAPI	Python backend framework for the chat API	https://fastapi.tiangolo.com/

LangChain RAG guide	Step-by-step RAG implementation tutorial	https://python.langchain.com/docs/tutorials/rag/
ChromaDB	Free local vector database for embeddings	https://www.trychroma.com/
Pinecone	Cloud vector database (free starter tier)	https://www.pinecone.io/
Ragas	RAG evaluation framework — measure answer quality	https://docs.ragas.io/en/latest/
Streamlit	Fast Python UI for demos	https://streamlit.io/
Vercel	Free frontend deployment	https://vercel.com/
Railway	Free backend deployment	https://railway.app/

RESUME BULLET POINTS

- Built a full-stack LLM assistant with streaming responses, session-based conversation memory, and tool use (web search, calculator) where the model autonomously decides when to call each tool.
- Implemented a RAG pipeline: chunked PDFs into 500-token segments, embedded with a text embedding model, stored in a vector database, and retrieved top-5 relevant chunks per query for grounded answers.
- Evaluated RAG quality using a dedicated evaluation framework — measuring faithfulness, context recall, and answer relevancy — and iterated on chunk size to improve scores.

INTERVIEW Q&A;

Q: What is RAG and why did you use it instead of fine-tuning?

A: RAG retrieves relevant documents at query time and injects them into the prompt as context. Fine-tuning bakes knowledge into model weights — expensive and still prone to hallucination. RAG is cheaper, updatable without retraining, and gives you source citations. For most production use cases, RAG is the right choice.

Q: How do you prevent hallucinations in your assistant?

A: Three layers: (1) RAG grounds answers in real retrieved documents. (2) The system prompt instructs the model to say 'I don't know' when context is insufficient. (3) An evaluation framework flags answers not supported by the retrieved context.

Q: How does streaming work technically?

A: The backend uses Server-Sent Events. The LLM SDK returns a stream object; I iterate over it and yield each token chunk as it arrives. The frontend uses the EventSource API to display tokens in real time instead of waiting for the full response.

04 — AI Safety & Evaluation Dashboard

A dashboard that tests AI model outputs for bias, toxicity, hallucination, and safety. Rare skill — AI governance is a legal requirement in 2025. High demand, low supply.

OVERVIEW

What it is	A tool that runs prompts through AI models and scores outputs on toxicity, bias, hallucination rate, and refusal rate using evaluation APIs and benchmarks. The EU AI Act (2025) requires AI systems to be evaluated and documented. Companies need engineers who can run these evals. Very few candidates have this skill.
Why it matters	
Difficulty	Intermediate — 2 weeks. No ML training required whatsoever.
Core concept	Test suite of prompts → run through model → score each response → display results in a dashboard with charts and comparison tables.

STEP-BY-STEP BUILD PLAN

Step 1	Define What You Measure	Choose 4 dimensions: (1) Toxicity — harmful or offensive language. (2) Bias — does the model respond differently to the same question about different groups? (3) Hallucination — does the model make up facts? (4) Refusal rate — does it refuse safe prompts?
Step 2	Build Your Test Suite	Create a CSV with 50 to 100 test prompts. Include benign prompts, adversarial prompts, and demographic-sensitive prompts. Use the public TruthfulQA and BBQ datasets as sources.
Step 3	Run Models and Collect Outputs	Write a Python script that sends each prompt to 2 or 3 different LLMs and stores: prompt, model name, response, and timestamp in a local database.
Step 4	Score Each Response	Send each response to a toxicity scoring API to get a score from 0 to 1. Use an LLM evaluation library for hallucination and relevancy scores. For bias: compare response quality scores across demographic groups in the prompts.
Step 5	Build the Dashboard	Show: overall safety score per model, breakdown by dimension, worst-performing prompts, and a side-by-side model comparison table. Use a charting library for bar charts and radar charts.
Step 6	Add Red-Teaming Feature	Let users type a custom adversarial prompt and instantly see its safety scores. This interactive feature is what makes the demo memorable in interviews.
Step 7	Deploy & Write a Report	Deploy on a free platform. Write a short report showing actual eval scores from your test runs with 2 to 3 models compared. Upload to GitHub with a detailed README.

TOOLS & REAL LINKS

Tool / Framework	Purpose	Link (type in browser)
------------------	---------	------------------------

DeepEval	LLM evaluation framework — hallucination, bias, toxicity	https://docs.confident-ai.com/
Google Perspective API	Toxicity scoring API (free, by Google Jigsaw)	https://perspectiveapi.com/
TruthfulQA dataset	Public benchmark for measuring hallucination	https://github.com/sylinrl/TruthfulQA
BBQ Bias Benchmark	Public dataset for measuring demographic bias	https://github.com/nyu-ml/BBC
LLM Eval frameworks list	Curated list of evaluation tools for LLMs	https://github.com/openai/evals
Ragas	Answer quality evaluation — faithfulness, context recall	https://docs.ragas.io/en/latest/
Streamlit Cloud	Free dashboard deployment	https://streamlit.io/cloud
EU AI Act summary	Regulatory context — mention this in interviews	https://artificialintelligenceact.eu/
Plotly Python	Charts and visualisations for the dashboard	https://plotly.com/python/

RESUME BULLET POINTS

- Built an AI Safety & Evaluation Dashboard that benchmarks LLM outputs across 4 safety dimensions (toxicity, bias, hallucination, refusal rate) using a toxicity API and a dedicated evaluation framework.
- Curated a 100-prompt adversarial test suite using TruthfulQA and BBQ Bias Benchmark datasets and automated scoring to compare 3 models side by side with real quantitative results.
- Designed a live red-teaming interface where users submit custom prompts and receive instant safety scores, demonstrating AI governance awareness aligned with EU AI Act evaluation requirements.

INTERVIEW Q&A;

Q: What is the EU AI Act and how does it relate to your project?

A: The EU AI Act (effective 2025) classifies AI systems by risk and mandates evaluation, logging, and transparency for high-risk systems. My dashboard directly addresses this — it produces quantitative safety scores and audit-ready logs for each model, which is exactly what compliance teams need.

Q: How do you measure bias in an AI model?

A: I use the BBQ dataset which contains questions where the correct answer should NOT depend on demographic attributes like race, gender, or age. I compare the model's accuracy and refusal rates across demographic groups. A biased model will give different quality answers to identical questions with different demographic framing.

Q: What's the difference between toxicity and safety?

A: Toxicity is a subset of safety. Toxicity specifically measures harmful, offensive, or hateful language — quantified by a toxicity API score. Safety is broader: it includes toxicity plus jailbreaks, factual misinformation, personal data leakage, and prompt injection vulnerabilities.

Recommended Build Order & Timeline

Build in this order. Each project adds skills used in the next one.

Weeks	Project	Key Milestone	Skill Gained
1–2	03 — LLM Assistant (basic)	Streaming chat API working	LLM API, backend basics
3	03 — Add RAG	Upload PDF, get grounded answers	Embeddings, vector DB
4–5	02 — Workflow Automation	3 live automations deployed	API chaining, AI in tools
6–7	01 — Multi-Agent System	3-agent graph solves a real task	LangGraph, orchestration
8–9	04 — Safety Dashboard	Dashboard with real eval scores	LLM evals, AI governance
10	All 4 — Portfolio Polish	README + Loom + GitHub live	Communication, presentation

PORTFOLIO CHECKLIST — DO THIS FOR EVERY PROJECT

README.md	Problem, architecture diagram image, setup steps, live demo link. If there is no README, the project does not exist to a recruiter.
Architecture diagram	Use draw.io (free): https://app.diagrams.net/ — export as PNG, add to README.
Live demo	Deploy every project. A dead or missing link = project doesn't exist.
Loom video	2-minute screen recording showing the project solving a real problem. Link it in the README.
Commit history	Real, meaningful commit messages. Interviewers look at this. Avoid a single massive initial commit.
Blog post (optional)	Write 1 post on Dev.to or a personal blog explaining what you built. Boosts discoverability and shows communication skills.

All links in this document are real. No fake data. Build in order, ship every project publicly, and document everything. These 4 projects, built and presented well, will get you shortlisted at any AI-focused company.